JavaScript


What is the world's most popular language?
A: French
B: English
C: Spanish
D: JavaScript

→ In one slide

- a weakly typed 🔗 dynamic language 🔗 with dynamic types 🔗 

```
let x; // x is undefined
console.log(x)
x = 5; // x is a Number
console.log(x)
x = 'John'; // x is a String
console.log(x)
```

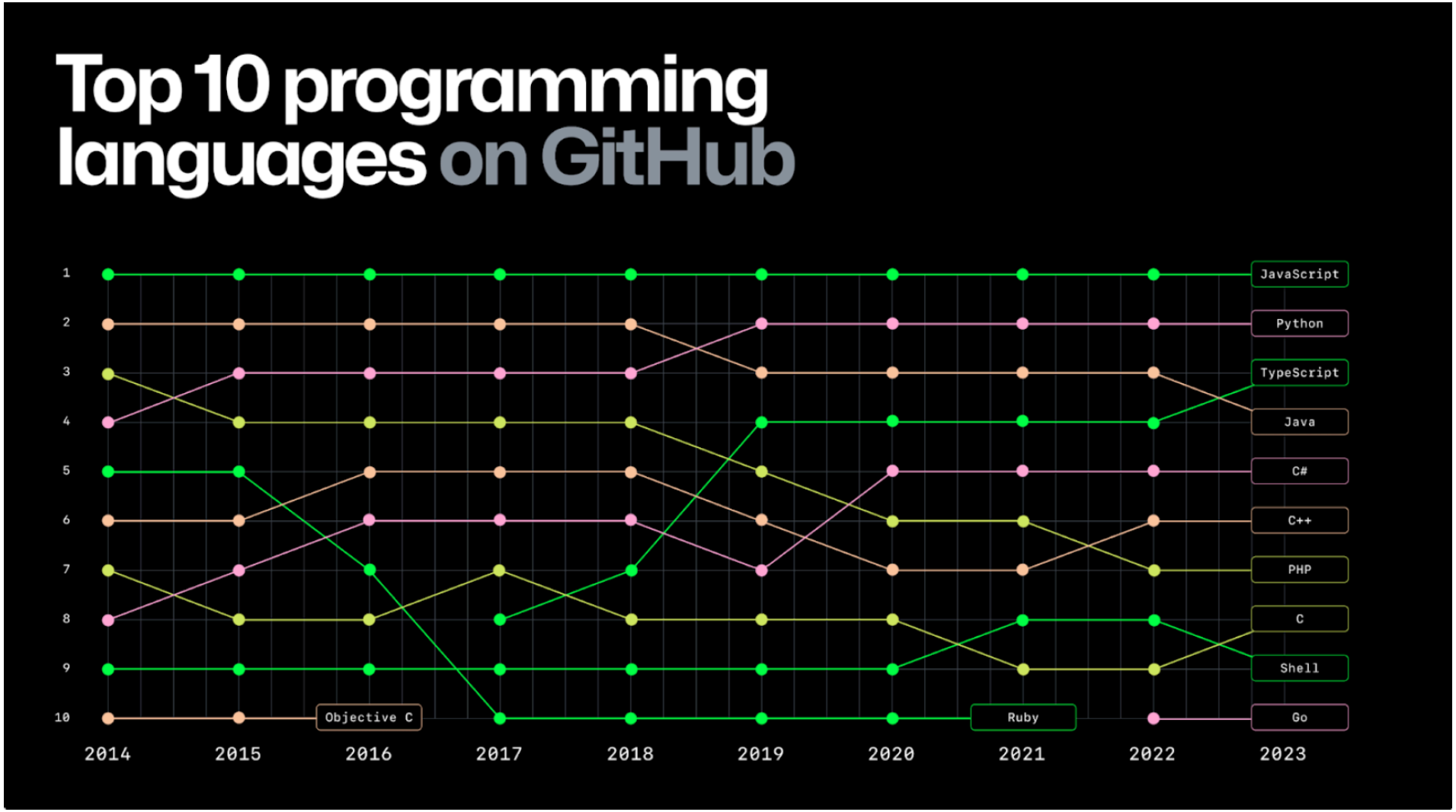
```
undefined
5
John
```

> ℹ️ weakly typed → allows implicit type conversion when an operation involves mismatched types, instead of throwing type errors

> ℹ️ Variables in JavaScript are not directly associated with any particular value type, and any variable can be assigned (and re-assigned) values of all types

> ⚠️ **Javascript can hold values with different data types in the same variable during runtime `[dynamic]`, and it can perform operations between different data types without explicitly converting them before `[weakly]`**

→ … however 💰



ℹ️ TypeScript → A typed superset of JavaScript

→ … however (again) 💰



Stack Overflow 2025 Developer Survey 🔗

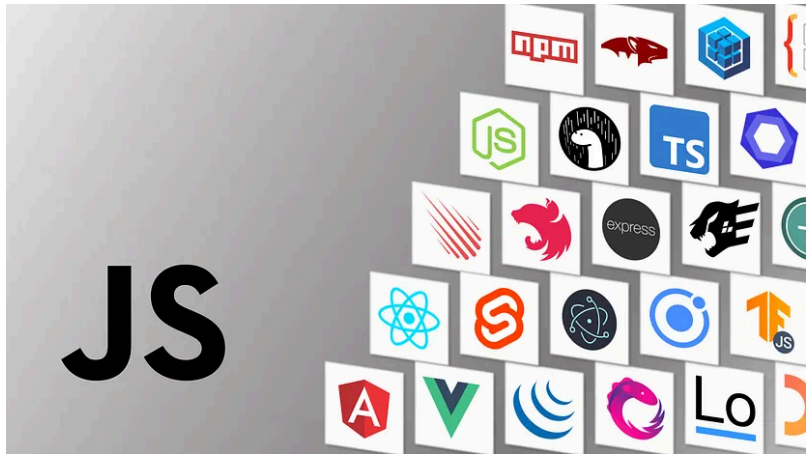
→ … however (x 3) 😄

Any application that can be written in JavaScript, will eventually be written in JavaScript

> Jeff Atwood, 2009
> Stack Overflow, Discourse

→ … however (x 4) 👑

- **core technology** of the World Wide Web
  - alongside HTML and CSS
- As of 2024, **98.8%** of websites use JavaScript on the client side for webpage behavior
- All major web browsers have a dedicated JavaScript engine to execute the code on users' devices
- **Huge Ecosystem**



→ History

- JavaScript was invented by Brendan Eich in 1995, and became an ECMA standard in 1997.
  - 1996 → Netscape 2 was released with JavaScript 1.0
  - 1997 → Internet Explorer (IE4) was the first browser to support ES1.
  - 2000 → Firefox 1 was released with JavaScript 1.5
- ECMAScript is the official name of the language.
- ECMAScript versions have been abbreviated to ES1, ES2, ES3, ES5, and ES6.
- Since 2016, versions are named by year (ECMAScript 2016, 2017, 2018, 2019, 2020).

> 🤼 Oracle, it's time to free JavaScript 🔗

→ History

- EcmaScript 1
  - The first version of the language standard.
- EcmaScript 2
  - standardized by ISO/IEC
- EcmaScript 3
  - Regular expression literals
  - New control statements
  - Exception handling
- EcmaScript 4
  - was abandoned in 2008 due to disagreements about its feature set
- EcmaScript 5
  - High-order iteration functions (map, reduce, filter, forEach)
  - JSON support
  - Getters and setters
  - Better reflection and object properties
- EcmaScript 6 (AKA ES6)
  - Promises
  - Modules
  - Classes
  - Block-scoped variable declarations (let)
  - Arrow functions
  - Template literal
  - ...

- ECMAScript version history 🔗

→ The JavaScript language

What can JavaScript Do?

- Websites
- Mobile Apps
- Game Development
- Artificial Intelligence
- Web Applications
- ...

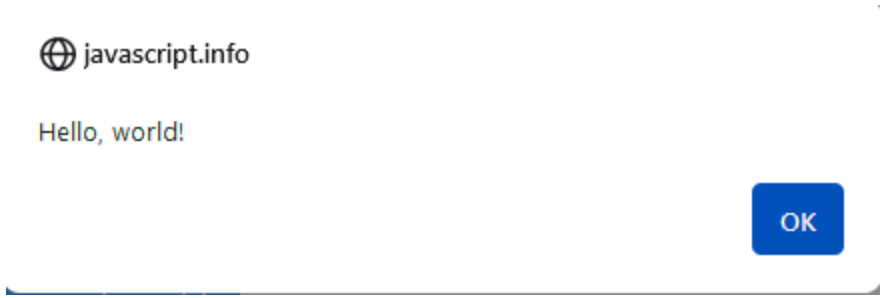
Strengths

- Easy setup
- Easy to Learn
- End-to-End Programming
- Huge Community Support
- Rich Interfaces
- Powerful Frameworks

Hello World: Client Side

- JavaScript code can be inserted almost anywhere into an HTML document using the "**script**" tag.

```
<!DOCTYPE HTML>
<html>
<body>
  <p>Before the script...</p>
  <script type="text/javascript"><!--
    alert( 'Hello, world!' );
  //--></script>
  <p>...After the script.</p>
</body>
</html>
```



Hello World: Server Side

- JavaScript code can be run on the server using a runtime environment

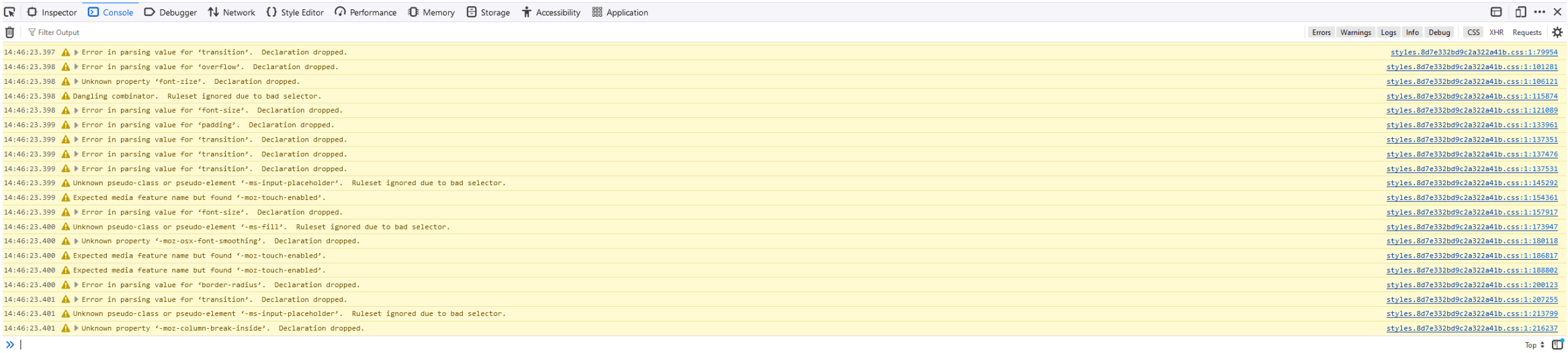


hello.js

```
console.log('hello world')
```

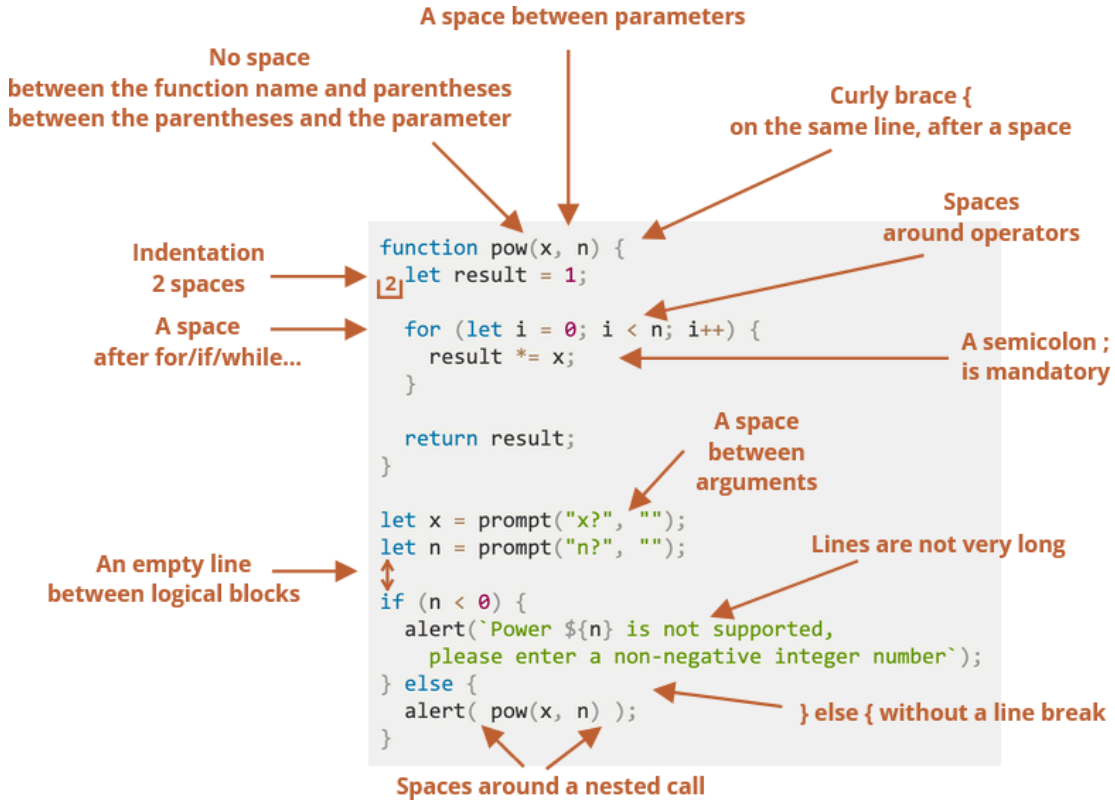
```
node hello.js
hello world
```

Developer console: Client Side



> ⌨ F12

Coding Style



suggested rules

- JavaScript Standard Style
- Google JavaScript Style Guide

Automated Linters

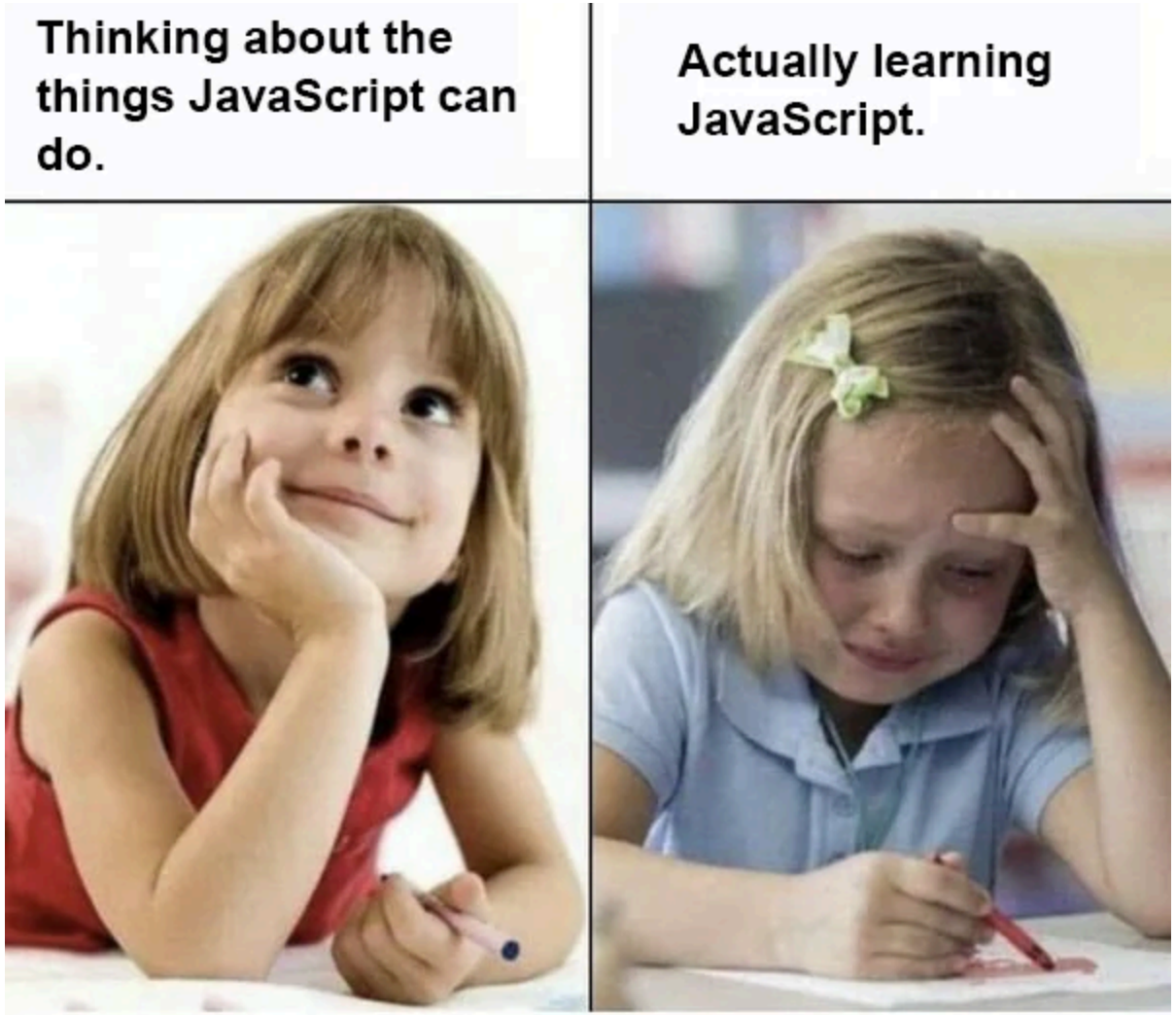
- jslint
- eslint

ℹ️ Linters are tools that can automatically check the style of your code and make improving suggestions

Required Refs

- Manuals 📖
  - MDN (Mozilla) JavaScript Reference🔗
- Specification 📄
  - ECMA-262🔗
    - A new specification version is released every year
- Compatibility tables 💔
  - JavaScript support among browser-based and other engines
  - compat-table🔗

→ Intro


Thinking about the things JavaScript can do.
Actually learning JavaScript.

- `+ - * / % **` → arithmetic
- `== != === !==` → equality test
- `< > <= >=` → comparison
- `&& || !` → logical
- `& | ^ ~ << >> >>>` → bitwise
- `= += &&= …` → assignment
- `?: ?? ?. , in …` → special
- `typeof instanceof void` → special

Data types

- primitive data types
  - **number** for numbers of any kind: integer or floating-point, integers are limited by $\pm(2^{53}-1)$
  - **bigint** for integer numbers of arbitrary length
  - **string** for strings.
    - A string may have zero or more characters, there's **no** single-**character type**
  - **boolean** for true/false
  - **null** for unknown values – a standalone type that has a single value null
  - **undefined** for unassigned values
    - a standalone type that has a single value undefined
- non-primitive data type
  - **object** for more complex data structures

- 1 `Number`

```
let n = 12.345;
```

  - both integer and floating point numbers
  - Infinity ∞ > than any number
    - `1 / 0`
  - NaN → a computational error
    - `"not a number" / 2`
    - NaN is sticky ❗

- 2 `BigInt`

```
let n = 1234567890123456789012345678901234567890n;
```

  - for numbers outside the $\pm(2^{53}-1)$ range

- 3 `Boolean`
  - logical type
  - `let isGreater = 4 > 1;`

4 `String`

- Double quotes: "Hello"
- Single quotes: 'goodbye'
- Backticks: `${str} and ${str2}`
-

```
let str = "Hello";
let str2 = 'goodbye';
let phrase = `${str} and ${str2}`;
let p1 = `${1+2} times ${str + " and " + str2}`;
let p2 = "${str} and ${str2}"; //⚠️
```

```
Hello
goodbye
Hello and goodbye
3 times Hello and goodbye
${str} and ${str2}
```

- Template literals → String interpolation
- **Immutable**
- UTF-16 characters
  - ```
    console.log("\u{1F3AF}");
    ```

    🎯
- String primitives VS String Objects
    ```

Backticks

```
let guestList = `Guests:
 * John
 * Pete
 * Mary
`;
console.log(guestList);
```

```
Guests:
 * John
 * Pete
 * Mary
```

Quotes

```
let guestList = "Guests:
  * John";
```

```
let guestList = "Guests:
                ^^^^^^^^^
SyntaxError: Invalid or unexpected token
```

Special characters

```
let guestList = "Guests:\n * John\n * Pete\n * Mary";
console.log(guestList);
let str1 = "Hello\nWorld";
let str2 = `Hello
World`;
console.log(str1 == str2);
```

```
Guests:
 * John
 * Pete
 * Mary
true
```

Common operations

```
console.log( `My\n`.length ); // length is property⚠️
let str = `Hello`;
console.log(str[0] );
console.log(str.at(0));
console.log(str[str.length - 1] );
console.log(str[-2] ); // undefined
console.log(str.at(-2) );
```

```
3
H
H
o
undefined
l
```

Iterations

```
for (let char of "Hello") {
  console.log(char);
}
```

```
H
e
l
l
o
```

Immutabillity

```
let str = 'Hi';
str[0] = 'h';
console.log( str[0] ); // doesn't work
```

```
H
```

Changing case

```
console.log( 'Interface'.toUpperCase() );
console.log( 'Interface'.toLowerCase() );
console.log( 'Interface'[0].toLowerCase() );
```

```
INTERFACE
interface
i
```

Searching for a substring

```
let str = 'Widget with id';
console.log( str.indexOf('Widget') );
console.log( str.indexOf('widget') );
console.log( str.indexOf('id') );
console.log( str.indexOf('id', 2) );
```

```
0
-1
1
12
```

Searching for a substring

```
let str = 'Widget with id';
if (str.indexOf('Widget')) {
    console.log('We found it');
}
if (str.indexOf('Widget') != -1) {
    console.log('We found it here');
}
console.log( str.includes('Widget') );
console.log( str.includes('wid', 3) );
console.log( str.startsWith('Wid') );
console.log( 'Widget'.endsWith('get') );
```

```
We found it here
true
false
true
true
```

Getting a substring

- `substring`, `substr` & `slice`
- `str.slice(start [, end])`
  - Negative values for start/end are also possible
- `str.substring(start [, end])`
  - start can be > than end
- `str.substr(start [, length])`
  - length instead of the ending position

Getting a substring

```
let str = 'stringify';
console.log( str.slice(0, 5) );
console.log( str.slice(2) );
console.log( str.slice(-4, -1) );
console.log( str.substring(2, 6) );
console.log( str.substring(6, 2) );
console.log( str.substr(2, 4) );
console.log( str.substr(-4, 2) );
```

```
strin
ringify
gif
ring
ring
ring
gi
```

ℹ️ Check the manual 🔗

Comparing strings

```
console.log( 'a' > 'Z' );
console.log( 'Österreich' > 'Zealand' );
let str = '';
for (let i = 65; i <= 220; i++) {
  str += String.fromCodePoint(i);
}
console.log( str );
```

```
true
true
ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`
abcdefghijklmnopqrstuvwxyz{|}~
 ¡¢£¤¥¦§¨©ª«¬®¯°±²³´µ¶·¸¹º»
¼½¾¿ÀÁÂÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖ×ØÙÚÛÜ
```

⚠️ for correct comparisons use localeCompare

- 5 `null`
  - > nothing, empty or value unknown 🤔
  - ```
    let age = null;
    console.log(age)
    ```
    ```
    null
    ```
- 6 `undefined`
  - > value is not assigned 🤔
  - ```
    let age;
    console.log(age)
    ```
    ```
    undefined
    ```
- 7 `Object`
  - > associative arrays with several special features 😰
  - properties → key-value pairs
  - To access a property
    - dot notation: `obj.property`
    - Square brackets notation `obj["property"]`

```
let user = { name: 'John', age: 30 };
console.log(user.name, user.age, user.hobby);
```

```
John 30 undefined
```

  - > ℹ️ unassigned properties are undefined

```
let user1 = new Object(); // "object constructor" syntax
user1.isAdmin = false;
let user2 = {};  // "object literal" syntax
user2['isAdmin'] = true;
console.log(user1.isAdmin, user2.isAdmin);
```

```
false true
```

```
delete user1['isAdmin'];
delete user2.isAdmin;
console.log(user1.isAdmin, user2.isAdmin);
```

```
undefined undefined
```

old-style → constructor function 🚫

```
function Person(name, age) {
  this.name = name;
  this.age = age;
  this.call = function(t) { … };
}
var p = new Person('John', 33);
```

> ℹ️ prototype method

new-style → class-based model ✅

```
class Person {
  constructor(name, age) {
    this.name = name;
     this.age = age;
  }
  call(t) { … }
}
let p = new Person("John", 33);
```

> 👍 use this style instead

Getters and setters

```
class Person {
  constructor(first, last) {
    this.first = first; this.last = last;
  }
  get name() {
    return `${this.first} ${this.last}`;
  }
  set name(n) {
    [this.first, this.last] = n.split(" ");
  }
}
```

```
let p = new Person("John", "Doe");
console.log(p.name);
p.name = "James Bond";
console.log(p.last);
```

```
John Doe
Bond
```

Static methods and properties

```
class Person {
  static totalWeight = 0;
  constructor(weight) {
    Person.totalWeight += weight;
  }
  static reportTotalWeight() {
    console.log(`The whole world weighs
      ${Person.totalWeight} kilos.`);
    }
}
```

```
let p1 = new Person(75);
let p2 = new Person(90);
Person.reportTotalWeight();
let p3 = new Person(80);
Person.reportTotalWeight();
```

```
The whole world weighs
        165 kilos.
The whole world weighs
        245 kilos.
```

Inheritance

```
class Person {
  constructor(first, last) {
    this.first = first; this.last = last;
  }
  get name() {
    return `${this.first} ${this.last}`;
  }
  set name(n) {
    [this.first, this.last] = n.split(" ");
  }
}
class Employee extends Person {
  constructor(first, last, salary) {
    super(first, last);
    this.salary = salary;
  }
  get name() {
    return `${super.name} (${this.salary})`;
  }
}
```

```
let e = new Employee("John", "Doe", 1000);
console.log(e.name);
```

```
John Doe (1000)
```

```
let obj1 = { a: 1, b: 2 };
let obj2 = { a: 1, b: 2 };
let obj3 = obj1;
console.log(obj1 == obj2);
console.log(obj1 === obj2);
console.log(obj3 == obj1);
console.log(obj3 === obj1);
```

```
false
false
true
true
```

> ℹ️ == and === operators compare object references, not their contents

Shallow VS Deep Comparison

> Shallow→ compares the top-level properties of the objects
> Deep→ compares the entire structure of the objects, including nested objects

```
/*Use this for plain objects*/
function compareObjects(obj1, obj2) {
  return JSON.stringify(obj1) === JSON.stringify(obj2);
}
console.log(compareObjects(obj1, obj2));
```

```
true
```

- **Loose** equality using `==`
  - performs type coercion before making any comparison

```
let a = 100;
let b = '100';
console.log(a == b)
let x = true;
let y = 'true';
console.log(x == y)
let z = true;
let w = 1;
console.log(z == w);
```

```
true
false
true
```

- **Strict** equality using `===`
  - does not convert the types of the operands before comparing

```
let a = 100;
let b = '100';
console.log(a === b)
let x = true;
let y = 'true';
console.log(x === y)
let z = true;
let w = 1;
console.log(z === w);
```

```
false
false
false
```

Computed properties 👴

```
let fruit = 'apple';
let bag = {
  [fruit + 'Computers']: 5
};
console.log(bag);
```

```
{ appleComputers: 5 }
```

Property value shorthand 🎖️

```
function makeUser(name, age) {
  return {
    name, // same as name: name
    age,  // same as age: age
  };
}
let user = makeUser('John', 30);
console.log(user);
```

```
{ name: 'John', age: 30 }
```

Property names limitations 🪄

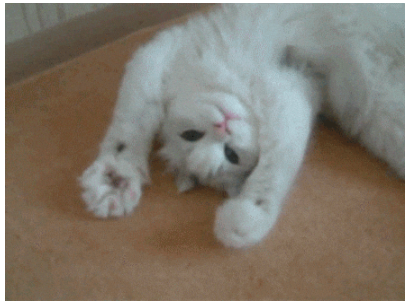
```
let obj = {
  for: 1,
  let: 2,
  return: 3
};
console.log( obj.for + obj.let + obj.return );
```

```
6
```

types are automatically converted to strings 🧙

```
let obj = {
  0: 'test'
};
console.log( obj['0'] , obj[0] );
```

```
test test
```



Property existence test

- `"in" operator`

```
let user = { name: 'John', age: 30 };
console.log('age' in user, user.noSuchProperty);
```

```
true undefined
```

iterate over all keys of an object

- `"key" in obj`

```
let user = {  name: 'John',  age: 30,  isAdmin: true };
for (let key in user) {
    console.log(key, user[key]);
};
```

```
name John
age 30
isAdmin true
```

Are objects ordered ?

```
let codes = {
  "49": "Germany",
  "41": "Switzerland",
  "44": "Great Britain",
  // ..,
  "1": "USA"
};
for (let code in codes) {
    console.log(code);
}
```

```
1
41
44
49
```

ℹ *integer* properties are sorted numerically

```
let user = {  name: 'John',   age: 30 };
user.isAdmin = true;
for (let prop in user) {
    console.log(prop);
}
```

```
name
age
isAdmin
```

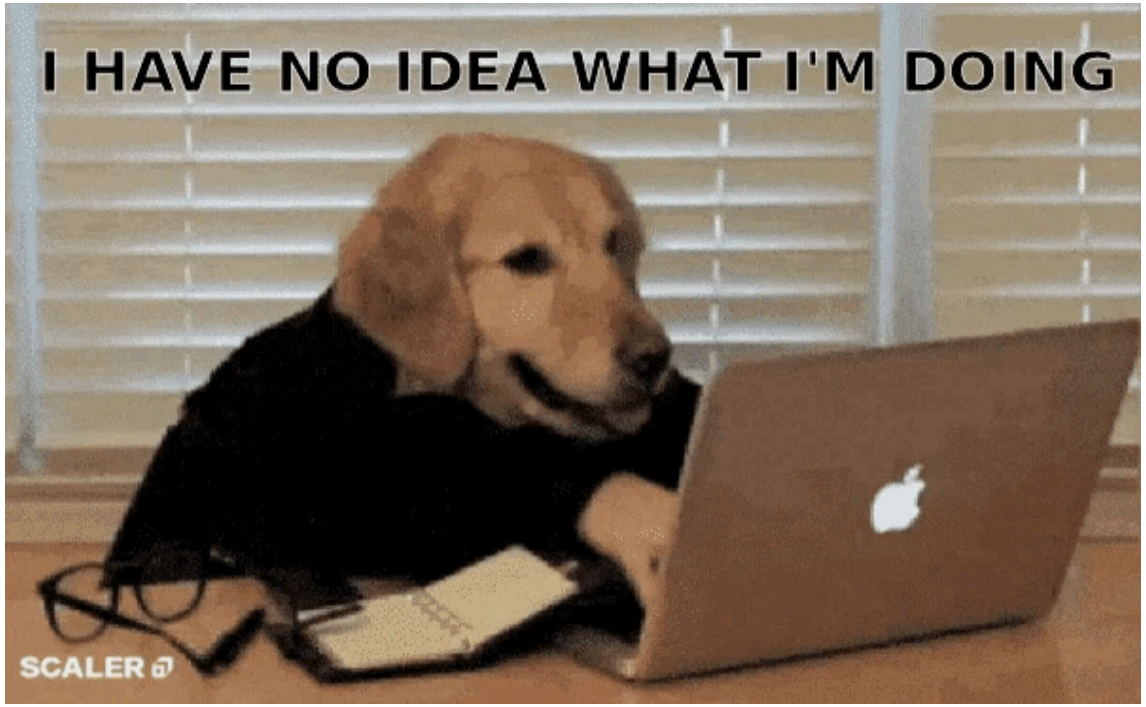
ℹ *non-integer* properties are sorted based on the creation order

? how can we fix the country codes

Like this 😄

```
let codes = {
  "+49": "Germany",
  "+41": "Switzerland",
  "+44": "Great Britain",
  // ..,
  "+1": "USA"
};
for (let code in codes) {
    console.log(+code);
}
```

```
49
41
44
1
```



ℹ Unary plus (+)

```
console.log('6' / '2');
let str = '123';
console.log(typeof str);
let num = Number(str); //explicit
console.log(typeof num);
console.log(Number('   123   '));
console.log(Number('123z'));
```

```
3
string
number
123
NaN
```

> ⚠️Numeric conversion → happens automatically

```
console.log(Number(true));
console.log(Number(false))
console.log(Number(undefined))
console.log(Number(null));
console.log(Boolean('hello'));
console.log(Boolean(''));
console.log(Boolean('0'));
console.log(Boolean(' '));
```

```
1
0
NaN
0
true
false
true
true
```

Object to primitive conversion

```
let user = {
    name: 'John',
    money: 1000,
    toString() {
        return `{name: "${this.name}"}`;
      },
    valueOf() {
        return this.money;
      }
  };
console.log(user);
console.log(String(user));
console.log(+user);
console.log(user + 500);
```

```
{
  name: 'John',
  money: 1000,
  toString: [Function: toString],
  valueOf: [Function: valueOf]
}
{name: "John"}
1000
1500
```

ℹ objects are auto-converted to primitives → the result of `obj1` ± `obj2` can't be another object!

⚠Symbol.toPrimitive

```
let user = { name: 'John',
    money: 1000,
    [Symbol.toPrimitive](hint) {
      console.log(`hint: ${hint}`);
      if (hint === 'number' ||
          hint === 'default') {
        return this.money;
      }
       if (hint === 'string') {
        return  `{name: "${this.name}"}`;
      }
    },};
console.log(user);
console.log(String(user));
console.log(JSON.stringify(user));
console.log(+user);
console.log(user.valueOf());
console.log(user + 10);
console.log(user * 2);
```

```
{
  name: 'John',
  money: 1000,
  [Symbol(Symbol.toPrimitive)]:
  [Function: [Symbol.toPrimitive]]
}
hint: string
{name: "John"}
{"name":"John","money":1000}
hint: number
1000
{
  name: 'John',
  money: 1000,
  [Symbol(Symbol.toPrimitive)]:
  [Function: [Symbol.toPrimitive]]
}
hint: default
1010
2000
```

The `typeof` operator

> typeof x → returns a string with the type name of the operand x

```
console.log(typeof undefined);
console.log(typeof 0 );
console.log(typeof 10n );
console.log(typeof true);
console.log(typeof 'foo');
console.log(typeof Symbol('id') );
console.log(typeof Math);
console.log(typeof null );
console.log(typeof console.log );
```

```
undefined
number
bigint
boolean
string
symbol
object
object
function
```

→ Intro → '??' & '?'

The `??` operator

> Nullish coalescing operator is commonly used to assign default values to variables

```
result = (a !== null && a !== undefined) ? a : b;
result = a ?? b
```

```
let firstName = null;
let lastName = null;
let nickName = 'Supercoder';
console.log(firstName ?? lastName
            ?? nickName ?? 'Anonymous');
```

```
Supercoder
```

The `?` operator

> Optional chaining '?.' operator stops the evaluation if the value before ?. is undefined or null and returns undefined

```
let user = {};
console.log(user.address.street); //⁉
console.log(user.address
            ? user.address.street
            ? user.address.street.name
            : null
            : null); // 🤫
```

```
let user = {};
console.log(user?.address?.street); // ✅
```

```
undefined
```

Optional Chaining → ?.() and ?.[]

```
let key = 'firstName';
let userAdmin = {
    firstName: 'John',
    admin() {
      console.log('I am admin');
    }
  };
let userGuest = {};
userAdmin.admin?.();
userGuest.admin?.();
console.log(userAdmin?.[key]);
console.log(userGuest?.[key]);
```

```
I am admin
John
undefined
```

- Variables named *apple* and *APPLE* are two different variables.
- Non-Latin letters are allowed

```
var μαθητής = 'σωκράτης'
var ΜΑΘΗΤΗΣ = 'σωκράτης'
console.log(μαθητής)
console.log(ΜΑΘΗΤΗΣ)
```

```
σωκράτης
σωκράτης
```

Declaration

- three ways of variable declaration:
- 1️⃣ let
- 2️⃣ const
- 3️⃣ ~~var~~ 🚫
  - ~~var~~ has no block scope

Scoping

```
if (true) { var test = true; }
console.log(test)
```

```
true
```

```
x = 5;
var x;
console.log(x)
```

```
5
```

> ℹ️ Hoisting → JavaScript's default behavior of moving all declarations to the top of the current scope
> JavaScript only hoists declarations, not initializations.

→ Intro → Variables & Scoping

```
y = 5;
let y;
```

```
y = 5;
  ^
ReferenceError: Cannot access 'y' before initialization
```

```
z = 10;
const z;
```

```
const z;
      ^
SyntaxError: Missing initializer in const declaration
```

```
function f() {
    let x = 17;
    if (true) {
      let x = 42;
      console.log(x);
    }
    console.log(x);
  }
f()
```

```
42
17
```

ℹ️ To avoid bugs 🐛, always declare all variables at the beginning of every scope.

use strict mode

```
x = 3.14;       // OK
myFunction();

function myFunction() {
  "use strict";
  y = 3.14;   // 🛑
}
```

> ℹ️ Strict mode 🔗

- several changes to normal JavaScript semantics
  - eliminates some JavaScript silent errors by changing them to throw errors
    - assigning to undeclared variables

```
function showMessage(from, text) {
    from = '*' + from + '*';
    console.log( from + ': ' + text );
  }
  let from = 'Ann';
showMessage(from, 'Hello');
console.log( from );
```

```
*Ann*: Hello
Ann
```

> a function always gets a copy of the value

```
function showMessage(from, text = 'no text given') {
    console.log( from + ': ' + text );
  }
showMessage('Ann');
showMessage('Ann', undefined);
```

```
Ann: no text given
Ann: no text given
```

```
// old - alternative method for default parameters
function showMessage(from, text) {
  text = text || 'no text given';
}
```

> If a function doesn't retrun a value → then its result is `undefined`

```
function myf(x, y, ...rest) {
  console.log(x, y, rest);
  console.log(arguments);
  for (let key in arguments) {
    console.log(typeof key, key);
  }
}
myf(1,2,3,4);
myf(1);
```

```
1 2 [ 3, 4 ]
[Arguments] { '0': 1, '1': 2, '2': 3, '3': 4 }
string 0
string 1
string 2
string 3
1 undefined []
[Arguments] { '0': 1 }
string 0
```

```
// Function Declaration
function sum1(a, b) {
    return a + b;
  }
//Function Expression
let sum2 = function(a, b) {
    return a + b;
};
let sum3 = sum2;
console.log(sum1(3,2));
console.log(sum2(3,2));
console.log(sum3(3,2));
```

```
5
5
5
```

> ℹ️ A **Function Expression** is created when the execution reaches it and is usable only from that moment

Callback functions

```
function ask(question, yes, no) {
    if (confirm(question)) yes();
    else no();
  }
  function confirm(question) {
    // some very complicated logic !
    return question.length % 2;
  }
  ask(
    'Do you agree?',
    function() { console.log('You said yes!'); },
    function() { console.log('You said NO!'); }
  );
```

```
You said yes!
```

```
let func = function(arg1, arg2, ..., argN) {
  return expression;
};
let func = (arg1, arg2, ..., argN) => expression;
```

```
let sum1 = (a, b) => a + b;
let double = n => n * 2;
let sum2 = (a, b) => {
  let result = a + b;
  return result;
};
let sayHi = () => console.log('Hi!');
console.log(sum1(3, 2));
console.log(double(3));
console.log(sum2(3, 2));
sayHi();
```

```
5
6
5
Hi!
```

⚠️ Arrow functions have no “this”

⚠️ Arrow functions can’t run with new

```
let group = {
    title: 'Our Group',
    students: ['John', 'Pete'],
    showList1() {
      this.students.forEach(function(student) { //🔴
        console.log(this.title + ': ' + student);
      });
    },
    showList2() {
      this.students.forEach(
        student => console.log(this.title + ': ' + student)
      );
    }
  };
group.showList1(); //🔴
group.showList2(); //✅
```

```
undefined: John
undefined: Pete
Our Group: John
Our Group: Pete
```

→ Intro → Functions → Closures

- imagine a simple Counter 🔢

```
let counter = 0;
function add() {
  counter += 1;
  console.log(counter);
}
add();
add();
add();
console.log(++counter); // 😢
```

```
1
2
3
4 //🔴
```

!?

```
function add() {
  let counter = 0;
  function plus() {counter += 1;}
  plus();
  return counter;
}
add().plus(); //❓
```

```
Uncaught TypeError: add(...).plus is not a function
```

```
const add = (function () {
  let counter = 0;
  return function () {
    counter += 1;
    console.log(counter);
    return counter
    }
})();
add();
add();
add();
```

```
1
2
3
```

```
let arr1 = new Array();
let arr2= [];
let fruits = ['Apple', 'Orange', 'Plum'];
console.log(arr1, typeof arr1);
console.log(arr2, typeof arr2);
console.log(fruits, typeof fruits);
console.log(fruits[2]);
fruits[2] = 'Banana';
fruits[3] = 'Lemon';
console.log(fruits)
```

```
[] object
[] object
[ 'Apple', 'Orange', 'Plum' ] object
Plum
[ 'Apple', 'Orange', 'Banana', 'Lemon' ]
```

ℹ Arrays → ordered collections (indexed by integers)

```
let arr = [ 'Apple',
          { name: 'John' },
            true,
            function() {
              console.log('hello');
              }
          ];
console.log(arr[1].name );
console.log(arr[arr.length-1] );
console.log(arr.at(1));
console.log(arr.at(-1));
arr[3]();
```

```
John
[Function (anonymous)]
{ name: 'John' }
[Function (anonymous)]
hello
```

ℹ Array → can store elements of any type

Copied by reference

```
let fruits = ['Banana'];
let arr = fruits;
console.log(arr === fruits);
arr[1] = 'Pear';
console.log(fruits);
```

```
true
[ 'Banana', 'Pear' ]
```

of any size

```
let matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
  ];
  console.log(matrix[1][1]);
  console.log(matrix.length);
  console.log(matrix[0].length);
```

```
5
3
3
```

Often misused 😒

```
let fruits = [];
fruits[3] = 5;
fruits.age = 25;
console.log(fruits);
console.log(fruits.length);
fruits.length = 2;
console.log(fruits);
delete fruits.age;
console.log(fruits);
console.log(fruits[0]);
```

```
[ <3 empty items>, 5, age: 25 ]
4
[ <2 empty items>, age: 25 ]
[ <2 empty items> ]
undefined
```

Queue → FIFO (First-In-First-Out)

```
let fruits = ['Apple', 'Orange', 'Pear'];
console.log(fruits.shift() );
console.log(fruits);
fruits.push('Banana');
console.log(fruits);
```

```
Apple
[ 'Orange', 'Pear' ]
[ 'Orange', 'Pear', 'Banana' ]
```

> ℹ️ shift → extracts the first element
> ℹ️ unshift → adds an element to the beginning

> which methods are 🐰 & which are 🐢 ?

Stack → LIFO (Last-In-First-Out)

```
let fruits = ['Apple', 'Orange', 'Pear'];
console.log(fruits.pop());
console.log(fruits);
fruits.push('Banana', 'Lemon');
console.log(fruits);
```

```
Pear
[ 'Apple', 'Orange' ]
[ 'Apple', 'Orange', 'Banana', 'Lemon' ]
```

> ℹ️ push → adds an element to the end
> ℹ️ pop → consumes an element from the end

Loops

```
let fruits = ['Apple', 'Orange', 'Pear'];
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
for (let fruit of fruits) {
   console.log(fruit);
} // NEW
for (let key in fruits) {
  console.log(fruits[key]);
} //
fruits.forEach(element => {
    console.log(element);
}); // NEW
```

```
Apple
Orange
Pear
...
```

toString 😵

```
let arr = [1, 2, 3];
console.log(arr);
console.log(String(arr) === '1,2,3');
console.log([] + 1); // [1] -> ""
console.log([1] + 1); // [1] -> "1"
console.log([1,2] + 1); // [1,2] -> "1,2"
```

```
[ 1, 2, 3 ]
true
1
11
1,21
```

```
console.log( [] == [] );
console.log( [0] == [0] );
console.log(  0 == [] );
console.log( '0' == [] );
```

```
false
false
true
false
```

> ℹ️ Compare array elements using an iterator🔗

`arr.splice(start[, deleteCount, elem1, ..., elemN])`

```
let arr = ['I', 'study', 'JavaScript',
           'right', 'now'];
let removed = arr.splice(0, 2);
console.log(arr);
console.log(removed);
removed = arr.splice(2, 0, 'complex', 'language');
console.log(arr);
console.log(removed);
removed = arr.splice(-1, 0, 'I', 'study');
console.log(arr);
console.log(removed);
```

```
[ 'JavaScript', 'right', 'now' ]
[ 'I', 'study' ]
[ 'JavaScript', 'right', 'complex',
  'language', 'now' ]
[]
[ 'JavaScript', 'right', 'complex',
  'language', 'I', 'study', 'now' ]
[]
```

other methods

```
let arr = ['t', 'e', 's', 't'];
let arr2 = arr.slice(1, 3);
console.log(arr);
console.log(arr2);
arr2 = arr.concat('i','n','g!')
console.log(arr2);
arr2 = arr.concat(' ' , arr);
console.log(arr2.length, arr2.indexOf(' '));
console.log(arr2.includes('t'));
```

```
[ 't', 'e', 's', 't' ]
[ 'e', 's' ]
[ 't',  'e', 's', 't',  'i', 'n', 'g!']
9 4
true
```

→ Intro → Arrays → transform

> `map(func)` → creates a new array from results of calling func for every element.

```
let lengths = ['Bilbo', 'Gandalf', 'Nazgul'].map(item => item.length);
console.log(lengths);
```

```
[ 5, 7, 6 ]
```

> `filter(func)` → creates a new array of all matching elements.

```
let users = [   {id: 1, name: 'Luke'},
  {id: 2, name: 'Zeb'},
  {id: 3, name: 'Ezra'}
];
let someUsers = users.filter(item => item.id < 3);
console.log(someUsers);
```

```
[ { id: 1, name: 'Luke' }, { id: 2, name: 'Zeb' } ]
```

> `reduce(func)` → calculate a single value based on the array.

```
let value = arr.reduce(function(accumulator, item, index, array) {
  // ...
}, [initial]);
```

```
let arr = [1, 2, 3, 4, 5];
let result = arr.reduce((sum, current) => sum + current, 0);
console.log(result);
```

```
15
```

> 1️⃣ initial → 0, current → 1 ➡️ accumulator → 1
> 2️⃣ initial → 1, current → 2 ➡️ accumulator → 3
> 3️⃣ initial → 3, current → 3 ➡️ accumulator → 6
> 4️⃣ initial → 6, current → 4 ➡️ accumulator → 10
> 5️⃣ initial → 10, current → 5 ➡️ accumulator → 15

→ Intro → Arrays → transform

```
var personnel = [

{ id: 25, name: 'Luke Skywalker',  pilotScore: 98,
  shootScore: 56, isForceUser: true,},

{ id: 82, name: 'Sabine Wren',  pilotScore: 73,
  shootScore: 99, isForceUser: false,},

{ id: 22, name: 'Zeb Orellios', pilotScore: 20,
  shootScore: 59, isForceUser: false,},

{ id: 15, name: 'Ezra Bridger', pilotScore: 43,
  shootScore: 67, isForceUser: true,},

{ id: 11, name: 'Caleb Dume', pilotScore: 71,
  shootScore: 85, isForceUser: true, },
];
```

get the total score of force users only ?



```
var personnel = [{
    id: 25,
    name: 'Luke Skywalker',
    pilotScore: 98,
    shootScore: 56,
    isForceUser: true, }
    ...
const totalJediScore = personnel
                            .filter(person => person.isForceUser)
                            .map(jedi => jedi.pilotScore + jedi.shootScore)
                            .reduce((acc, score) => acc + score, 0);
console.log(totalJediScore);
```

```
420
```

> 📑 Simplify your JavaScript – Use .map, .reduce and .filter 🔗

→ Intro → Map

- ℹ️ Map is a collection of keyed data items

```
let map = new Map();
map.set('1', 'str1');
map.set(1, 'num1');
map.set(1, 'num2');
map.set(true, 'bool1');
console.log(map.get(1));
console.log(map.get('1'));
map.set({ name: 'John' }, 123);
console.log(map.get({ name: 'John' }));
console.log(map.has(1));
console.log(map.delete(1));
console.log(map.delete('str1'));
console.log(map.get(1));
console.log(map.size);
```

```
num2
str1

undefined
true
true
false
undefined
3
```

- `map.keys()`, `map.values()`, `map.entries()`, `forEach`

```
let recipeMap = new Map([
  ['cucumber', 500],
  ['tomatoes', 350],
  ['onion',    50]
]);
for (let vegetable of recipeMap.keys()) {
  console.log(vegetable);
}
for (let amount of recipeMap.values()) {
  console.log(amount);
}
for (let entry of recipeMap) {
  console.log(entry);
}
recipeMap.forEach( (value, key, map) => {
  console.log(`${key}: ${value}`);
});
```

```
cucumber
tomatoes
onion
```

```
500
350
50
```

```
[ 'cucumber', 500 ]
[ 'tomatoes', 350 ]
[ 'onion', 50 ]
```

```
cucumber: 500
tomatoes: 350
onion: 50
```

ℹ Map from Object → `Object.entries()`

```
let obj = {
  name: 'John',
  age: 30
};
let map = new Map(Object.entries(obj));
console.log(map.get('name'));
console.log(map.get('age'));
```

```
John
30
```

ℹ Object from Map → `Object.fromEntries()`

```
let prices = Object.fromEntries([
  ['banana', 1],
  ['orange', 2],
  ['meat', 4]
]);
console.log(prices.banana);
console.log(prices.orange);
```

```
1
2
```

```
let recipeMap = new Map([
  ['cucumber', 500],
  ['tomatoes', 350],
  ['onion',    50]
]);
let recipeMapObj = Object.fromEntries(recipeMap);
console.log(recipeMapObj.cucumber);
let recipeMapObj2MAP = new Map(Object.entries(recipeMapObj));
console.log(recipeMapObj2MAP.get('cucumber'));
```

```
500
500
```

→ Intro → Set

> ℹ Set is a collection of values without dublicates

```
let set = new Set();
let john = { name: 'John' };
let pete = { name: 'Pete' };
let mary = { name: 'Mary' };
set.add(john);
set.add(pete);
set.add(mary);
set.add(john);
set.add(mary);
console.log(set.size);
for (let user of set) {
  console.log(user.name);
}
set.delete(john);
console.log(set.has(john));
```

```
3
John
Pete
Mary
false
```

- `for..of`, `forEach`
  - `set.keys()` (`set.values()`, `set.entries()`)

```
let set = new Set(['oranges', 'apples', 'bananas']);

for (let value of set)
  console.log(value);

set.forEach((value, valueAgain, set) => {
  console.log(value);
});
```

```
oranges
apples
bananas
oranges
apples
bananas
```

Callbacks → typical for timeouts and listening to events

```
setTimeout(() => { doSomething() }, 2000);
```

```
function myDisplayer(some) {
  document.getElementById('demo').innerHTML = some;
}
function myCalculator(num1, num2, myCallback) {
  // imagine a time-consuming task
  let sum = num1 + num2;
  myCallback(sum);
}
myCalculator(5, 5, myDisplayer);
```

Promise → contains both the producing code and calls to the consuming code

> Callback

```
function myDisplayer(some) {
  document.getElementById("demo").innerHTML = some;
}
function getFile(myCallback) {
  let req = new XMLHttpRequest();
  req.open('GET', "mycar.html");
  req.onload = function() {
    if (req.status == 200) {
      myCallback(req.responseText);
    } else {
      myCallback("Error: " + req.status);
    }
  }
  req.send();
}
getFile(myDisplayer);
```

> Promise

```
function myDisplayer(some) {
  document.getElementById("demo").innerHTML = some;
}
let myPromise = new Promise(function(myResolve, myReject) {
  let req = new XMLHttpRequest();
  req.open('GET', "mycar.htm");
  req.onload = function() {
    if (req.status == 200) {
      myResolve(req.response);
    } else {
      myReject("File not Found");
    }
  };
  req.send();
});
myPromise.then(
  function(value) {myDisplayer(value);},
  function(error) {myDisplayer(error);}
);
```

> Promise states → pending,fulfilled, rejected

→ Intro → Asynchronous programming hell 🔥



```
console.log("Pyramid of doooooooooom!😨😵")

step1(function (value1) {
    step2(function (value2) {
        step3(function (value3) {
            step4(function (value4) {
                step5(function (value5) {
                    step6(function (value6) {
                        step7(function (value7) {
                            //Do something with value 4
                            console.log(value7)
                        });
                    });
                });
            });
        });
    });
});
```

Callback Hell 😢

```
fetchRandomJoke((joke) => {
    console.log(joke);
    translateJoke(joke, (translatedJoke) => {
        console.log(translatedJoke);
        postJoke(translatedJoke, () => {
            console.log("Joke posted successfully!");
        });
    });
});
```

Promise Hell 😢

```
connectDatabase()
  .then((database) => {
    return findAllBooks(database)
      .then((books) => {
        return getCurrentUser(database)
          .then((user) => {
            return pickTopRecommendation(books, user);
          });
      });
  });
```

→ Intro → Asynchronous programming

> ℹ `aync` → a function returns a promise
> ℹ `await` → wait until that promise settles and returns its result

```
async function getFile() {
  let myPromise = new Promise(function(resolve) {
    let req = new XMLHttpRequest();
    req.open('GET', "mycar.html");
    req.onload = function() {
      if (req.status == 200) {
        resolve(req.response);
      } else {
        resolve("File not Found");
      }
    };
    req.send();
  });
  document.getElementById("demo").innerHTML = await myPromise;
}
getFile();
```

→ Modules

- One script is one module
  - it contains a class or a library of functions for a specific purpose
- `export` → labels variables and functions that should be accessible from outside the current module.
- `import` → allows the import of functionality from other modules.

```
// 📁 sayHi.js
export function sayHi(user) {
console.log(`Hello, ${user}!`);
}
```

```
// 📁 main.js
import {sayHi} from './sayHi.js';
sayHi('John'); // Hello, John!
```

> Modules always work in strict mode
> Each module has its own top-level scope
> A module code is evaluated only the first time when imported

Typescript ⊃ JavaScript

- TypeScript provides static typing on top of JavaScript
  - Types by Inference or Declared

```
const personDescription = (name, city, age) =>
  `${name} lives in ${city}. he's ${age}. In 10 years he'll be ${age + 10}`;
console.log(personDescription("Germán", "29", "Buenos Aires"")); // 😢
```

VS

```
const personDescription = (name: string, city: string, age: number) =>
  `${name} lives in ${city}. he's ${age}. In 10 years he'll be ${age + 10}.`;
console.log(personDescription("Germán", "29", "Buenos Aires"")); // 🛑
```

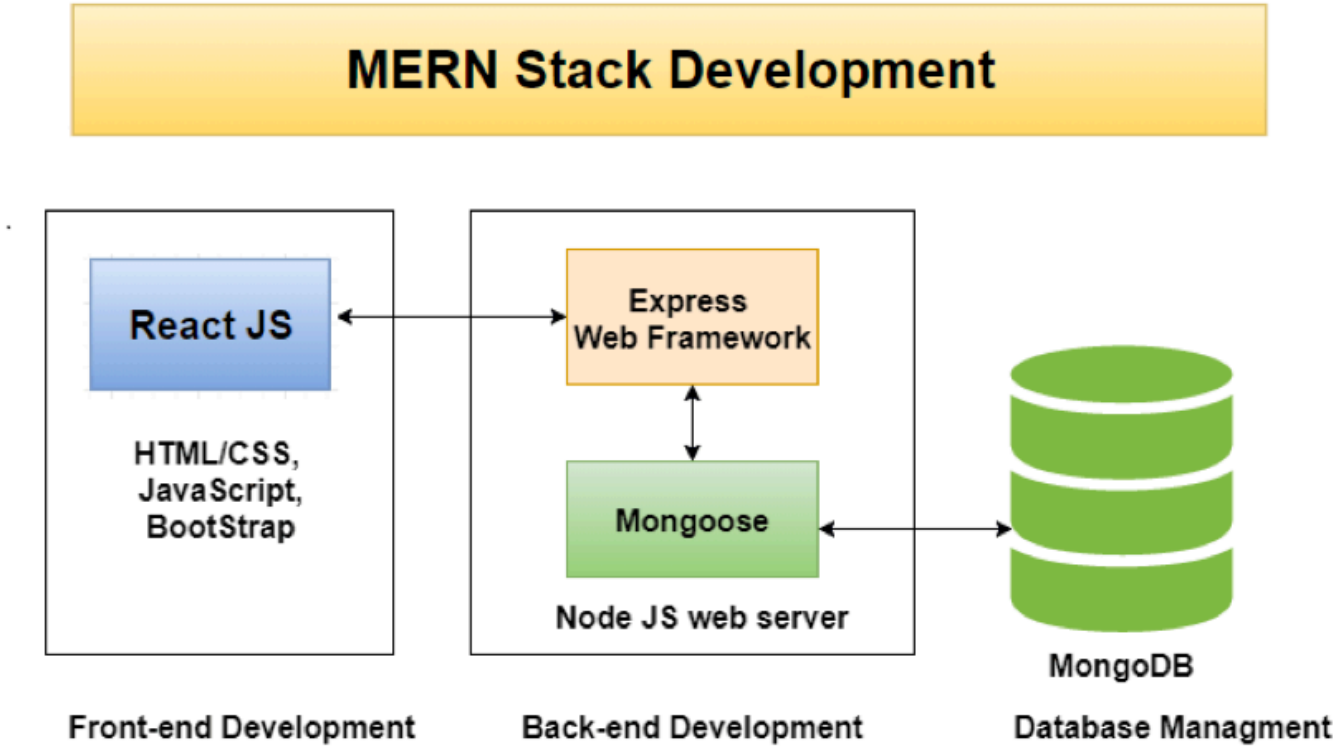
- TypeScript checks types through its compiler & transpiles code into JavaScript

→ Ecosystem

> vast collection of tools, frameworks, and libraries

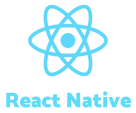
- **Front-end** development
  - frameworks → pre-structured JavaScript codes that provide a foundation for building the user interface of a web application 
  - libraries → DOM Manipulation, Data Handling, UI Components 
- **Back-end** development
  - frameworks → routing, db integration, & server-side logic 

→ Ecosystem


MERN Stack Development
React JS
HTML/CSS,
JavaScript,
BootStrap
Express
Web Framework
Mongoose
Node JS web server
MongoDB
Front-end Development
Back-end Development
Database Managment


M
E
R
N

→ Ecosystem

- Command line interface (CLI) applications
  - Package managers - Build Automation Tools
- Desktop (GUI) applications 
- Mobile applications 

→ Thank you!

Ερωτήσεις ?

